

Especificaciones del «Proyecto Final»

Fundamentos de Sistemas Embebidos

Autor: José Mauricio Matamoros de Maria y Campos

Simplicity is prerequisite for reliability.

—Edsger W. Dijkstra

1. Instrucciones

Se dice que sólo quien domina un tema podrá guiar a otros en el camino de su aprendizaje. Es así que, mediante el diseño y elaboración de un tutorial, el alumno probará que ha adquirido conocimientos robustos sobre cómo operan los sistemas embebidos, así como de sus características y limitaciones.

El tutorial estará acompañado de un prototipo funcional realizado de forma individual o por un equipo de hasta 3 (tres) personas y que será presentado al profesor¹ en la fecha convenida junto con todos los entregables solicitados; a saber: i) reporte escrito tipo práctica o tutorial (individual), ii) prototipo funcional, iii) video-evidencia y iv) enlace al repositorio con el código fuente del proyecto.

Los entregables se entregarán vía Classroom en la asignación pertinente antes de la fecha límite convenida y siguiendo las especificaciones de entrega detalladas en la [Sección 6](#)

Los proyectos propuestos (véase la [sección 5](#)) son los siguientes:

- [Centro Multimedia](#)
- [Consola de videojuegos *Retro*](#)
- [Control de Invernadero](#)
- [Concentrador básico *Smart Home*](#)
- [Distro de propósito específico](#)

Otros proyectos son aceptables previo acuerdo con el profesor de la asignatura.

2. Reporte escrito

El reporte escrito estará elaborado a manera de práctica de laboratorio o tutorial. El reporte elaborado asumirá que su lector no está familiarizado con la teoría de los sistemas embebidos y que su conocimiento de electrónica digital es básico, por lo que deberá guiarle paso a paso en la implementación de una solución práctica desde el alambrado de los componentes electrónicos, su conexión con el microcontrolador, la configuración de la tarjeta microcontroladora y el desarrollo los programas operativos y de control, hasta la integración de todos los componentes de software y hardware considerando la solución de problemas comunes; todo de manera simple, concisa y explicativa.

El tutorial deberá estar estructurado de manera similar a una práctica de laboratorio con antecedentes teóricos y deberá incluir, entre otros puntos:

- Objetivo
- Lista de materiales o componentes electrónicos, incluyendo posibles equivalentes si así fuere necesario
- Descripción del funcionamiento de los componentes electrónicos relevantes (circuitos integrados, encapsulados, controladores, etcétera)
- Descripción del funcionamiento de la tarjeta controladora (ej. Raspberry Pi) o circuito microcontrolador (ej. Microchip PIC) a utilizar, así como de los componentes relevantes de los mismos

¹Se espera que los programas hayan sido probados de antemano y que estos funcionen adecuadamente, ya sea en la tarjeta controladora con los periféricos alambrados (deseable), en la tarjeta controladora con señales simuladas o en un simulador.

- Información sobre el cuidado de la salud y advertencias de riesgos (ej. quemaduras, electrocución, etcétera) según corresponda
- Información sobre el cuidado de los componentes electrónicos delicados (ej. memorias) que así lo requieran, tales como uso de pulsera anti-estática, acumulación de grasa en terminales, límites de temperatura, etcétera
- Configuración de la tarjeta controladora (ej. Raspberry Pi) o circuito microcontrolador (ej. Microchip PIC) a utilizar, incluyendo puertos, periféricos, configuraciones del sistema operativo, paquetes y librerías a instalar, etcétera
- Desarrollo de los componentes de software (módulos) para cada una de las funciones que desempeñará la tarjeta
- Integración de los componentes de software o módulos en una solución de software
- Conclusiones
- Cuestionario
- Bibliografía de consulta

Al ser de carácter individual, deberá enfocarse especialmente en los aspectos desarrollados por cada alumno (ej. software, hardware, algún módulo en particular, etcétera), sin perder de vista que se presenta un tutorial que permite a un usuario novato replicar los resultados obtenidos.

Asimismo, el reporte escrito deberá cumplir las siguientes especificaciones técnicas:

- **SIN CARÁTULA**
- Sin escudos de la UNAM, FI o cualquier otro logotipo oficial
- Formato PDF.
- Se prefiere el uso de L^AT_EX con la plantilla `article` (hoja carta, fuente de 12pt).
- Si se usa procesador de palabras compatible (ej. Libreoffice Writer), use fuente tipo serif con tamaño entre 10.5 y 12 puntos. Se aconseja Times New Roman.
- Espaciado sencillo
- Márgenes cuadrados de 2.5cm
- Páginas foliadas al pie con alineación derecha (Página X de Y)
- Las figuras y tablas deberán estar numeradas y tener su pie de foto o título según corresponda.
- El documento electrónico deberá contener un enlace al video demostrativo en línea (ej. Youtube).
- El documento electrónico deberá contener un enlace al repositorio con el código fuente del proyecto (ej. GitHub).
- El trabajo escrito es de carácter individual.

Nota: El desarrollo del proyecto podrá realizarse en equipos de hasta 3 personas, mientras que los tutoriales serán realizados y entregados de manera individual.

3. Video-evidencia

Se entregará un video demostrativo por equipo donde pueda observarse el proyecto (o su simulación) en operación. Se deberá tener especial cuidado en que el video muestre cada uno de los requisitos listados a satisfacción. Es decir, no deberá dejar lugar a duda sobre si la característica fue implementada o no.

Preferentemente el video deberá subirse a YouTube o Vimeo y el enlace al mismo deberá proporcionarse en un archivo de texto plano (`txt`) en la carpeta *vid* del entregable empaquetado, así como en el reporte escrito y en el archivo (`readme.md`). Además, el video deberá tener las siguientes características:

- El video debe ser público y estar listado.
- El video debe tener audio.
- El video debe tener una resolución de al menos 720p.

Nota: Otros medios de almacenamiento como Dropbox, Google Drive, MEGA, etcétera, no son aceptables.

Si el video-evidencia se proporciona como archivo adjunto, éste deberá tener la siguientes características técnicas.

- Resolución no menor a 720×480 px
- Video codec MPEG-4 Part 10 (codec H.264/AVC)
- Audio codec: AAC 22050kHz 16bit mono o superior

4. Repositorio y código fuente

Se entregará un enlace a un repositorio público por equipo (por ejemplo Gitlab, Bitbucket o GitHub) de donde pueda de donde pueda descargarse el código fuente de los programas implementados para el proyecto, así como los archivos de configuración automatizada y/o documentos técnicos pertinentes.

Dicho enlace deberá incluirse tanto en el reporte escrito como en el archivo (`readme.MD`) del entregable empaquetado, incluyendo instrucciones simples y específicas para su descarga del servidor remoto. Además, se incluirá una copia del código fuente en la carpeta `src` del entregable empaquetado.

En particular se espera que junto con el código fuente se incluya un archivo de scripting (*secure shell* o *bash*) que realice de forma automatizada la compilación, instalación y configuración del proyecto completo en una instalación limpia de Raspbian.

Importante: Los archivos de código fuente deberán estar debidamente documentados, incluyendo el nombre del autor y la licencia original.

IMPORTANTE: No se aceptan proyectos que utilicen las simulaciones implementadas para las prácticas.

5. Proyectos

En esta sección se enlistan los proyectos aceptables como proyectos finales. Otros proyectos podrán ser propuestos por el alumno siempre y cuando cumplan con requisitos de dificultad e integración equivalentes.

En ningún caso los proyectos podrán usar distribuciones Linux con escritorio ni se podrá hacer uso de teclado y mouse. Se espera que el microcontrolador esté configurado para ejecutar de manera automática los programas necesarios para llevar a cabo su función embebida tan pronto se suministre alimentación al dispositivo y se termine el proceso de carga.

5.1. Control de Invernadero

El alumno implementará un control que permite monitorear y administrar remotamente un invernadero. El sistema embebido descrito deberá realizar las siguientes funciones:

- Encendido y apagado de sistema de irrigación
- Despliegado de gráfica con histórico de temperatura, irrigación y acciones tomadas
- Control de temperatura del invernadero utilizando PID
- Control de potencia del radiador (foco incandescente)
- Control de potencia del ventilador (ventilador de CPU o cualquier otro motor DC)
- Programado de ciclos de temperatura e irrigado
- El sistema embebido incorporará un servidor web el control remoto (local vía IP) de las funciones descritas anteriormente.

El prototipo presentado consiste en un volumen cerrado de al menos 20dm^3 (ej. una pecera) con al menos dos sensores de temperatura, en diferentes puntos. El radiador enciende cuando la temperatura es demasiado baja, mientras que el ventilador cumple con dos funciones: mantener un flujo de aire constante para conseguir una temperatura homogénea e inyectar aire fresco del exterior si la temperatura sube demasiado (pueden ocuparse dos ventiladores). El sistema de irrigación consiste en una bomba de agua de baja potencia (ej. de acuario) o una electroválvula.

Opcionalmente se podrá incorporar un sensor de humedad para controlar los ciclos de irrigación de manera automática, así como una matriz de leds para complementar los ciclos día/noche en el invernadero.

Las gráficas con el historial de temperatura y acciones tomadas (encendido de ventilador, sistema de irrigación, etcétera) deberán guardarse en almacenamiento secundario para estar disponibles cada vez que se acceda al sistema de forma remota.

El componente clave de este proyecto es el control de una carga resistiva de alta potencia (el foco incandescente) usando un controlador PID; por lo que sin éste componente el proyecto no será aceptado. El control de la velocidad del ventilador se realizará mediante un controlador de potencia operado con una señal PWM.

5.2. Concentrador básico *Smart Home*

El alumno implementará un concentrador que permite monitorear y administrar remotamente via navegador los dispositivos inteligentes domésticos vinculados.

El sistema embebido descrito deberá realizar las siguientes funciones:

- Despliegado de cámaras de vigilancia conectadas²
- Encendido y apagado de luces (al menos una)
- Atenuado de luces (al menos una)
- Detección de timbre de puerta (al menos una)
- Apertura remota de la puerta de la cochera
- Programado de encendido y apagado de luces e interruptores
- El sistema embebido incorporará un servidor web el control remoto (local vía IP) de las funciones descritas anteriormente.

El prototipo presentado consiste en la maqueta de una casa o departamento (casa de muñecas) conectada a la línea de AC. Desde la aplicación remota, el usuario podrá encender o apagar las lámparas de la casa (al menos una es atenuable), controlar la puerta de la cochera y observar la transmisión de video de las cámaras IP conectadas al sistema. Asimismo, el usuario recibirá una alerta cuando se toque el timbre y, mediante una cámara, podrá observar quién llama a la puerta.

Para este proyecto hay dos rutas viables. Podrán utilizarse componentes comerciales que utilicen algún protocolo abierto conocido con el que la Raspberry Pi u otro microcontrolador pueda interactuar (ej. *Ethernet over Power*, *openHAB* o *Home Assistant*), o bien se podrán presentar componentes de factura propia. En ningún caso podrán utilizarse aplicaciones comerciales u *open source* como interfaz de aplicación de usuario.³

El componente clave de este proyecto es el despliegado del feed de video de las cámaras al presionar el timbre (interrupción); por lo que sin éste componente el proyecto no será aceptado. Todos los componentes, incluyendo el timbre y el motor de la cochera, deben operar con la línea de AC.

Importante: Push-buttons, LEDs, servomotores y otros componentes de DC no son aceptables en este proyecto.

5.3. Centro Multimedia

El alumno implementará un centro de entretenimiento para la reproducción de películas, videos, música, y fotografías.

El sistema embebido descrito deberá realizar las siguientes funciones:

- Asistente para conexión a internet [opcional]
- Menú al encendido que permitirá elegir entre servicios en línea para video (ej. Netflix, HBO-Go, Blim), música (ej. Spotify, Cloudfare, Deezer, etc) o reproducir el medio extraíble.
- Cuando se inserte un medio extraíble (memoria USB) con fotografías, estas serán reproducidas en modo presentación.
- Cuando se inserte un medio extraíble (memoria USB) con música, las pistas serán reproducidas una tras otra en bucle infinito.
- Cuando se inserte un medio extraíble (memoria USB) con videos, deberá poderse elegir qué película reproducir o si los videos serán reproducidos uno tras otro en modo presentación.
- Cuando se inserte un medio extraíble (memoria USB) con contenido mixto el aparato deberá preguntar qué acción tomar.
- Soporte para control remoto (apuntador y teclado en pantalla es aceptable pero no recomendado)

La tarjeta controladora se configurará de tal manera que al ser conectada a un televisor ésta proveerá capacidades de *SmartTV* a la pantalla conectada. El dispositivo no sólo permite disfrutar de las opciones de entretenimiento en línea, sino que permite al usuario reproducir películas, música y otros multimedios propios almacenados en medios extraíbles de forma inteligente, por lo que rápidamente revisará el contenido del medio antes de sugerir una acción a llevar a cabo.

²El sistema tendrá que simular dos cámaras IP por medio de la difusión o *streaming* de dos videos cortos (320×240@15fps).

³Nada de *apps* de la Google Store, App Store, descargables, etc. El software es de factura propia, el hardware no necesariamente.

La interfaz presentada al usuario debe ser limpia, amigable, fácil de usar y libre de ventanas; con instrucciones claras y precisas sobre el uso de las opciones que aparecen en pantalla.

El componente clave de este proyecto es la reproducción de video protegido con DRM; por lo que sin éste componente el proyecto no será aceptado. El video-evidencia debe mostrar claramente que la Raspberry Pi es capaz de decodificar video protegido transmitido por streaming.

Importante: El uso de distribuciones de escritorio o manejadores de ventanas superpuestas en este proyecto será penalizado severamente.

Importante: La implementación del proyecto debe ser realizada desde un sistema operativo base. El uso de proyectos existentes como *Kodi*, *KDE Plasma Bigscreen*, *Plex*, *Chromecast*, *Android TV*, etcétera significan reprobación automática.

5.4. Consola de videojuegos *Retro*

El alumno implementará un centro de entretenimiento emulación de una o más consolas de videojuegos usando higan, xMAME, Mednafen o Snes9x.

El sistema embebido descrito deberá realizar las siguientes funciones:

- Reproducción de animación o imagen estática (ej. logotipo) personalizada con sonido durante el arranque (secuencias de comandos o animaciones de arranque de Raspbian/Ubuntu no son aceptables).
- Arranque directo a emulador (sin escritorio).
- Control completo de la consola usando gamepad o joystick (no teclado, no mouse).
- Set de al menos 15 ROMS precargadas.
- Cuando se inserte un medio extraíble (memoria USB) con ROMS, el sistema pausará la simulación, copiará las ROMS a la tarjeta de almacenamiento local y mostrará al usuario la lista de ROMS disponibles.
- Soporte para control remoto (apuntador y teclado en pantalla es aceptable)

La tarjeta controladora se configurará para operar como una consola de videojuegos al conectarse a una pantalla, presentando al usuario la galería de juegos de 8 y 16 bits disponibles. El dispositivo además permite la expansión de la biblioteca de juegos copiando aquellos que están almacenados en medios extraíbles a la memoria interna de forma inteligente y evitando duplicados.

La interfaz presentada al usuario debe ser limpia, amigable, fácil de usar y libre de ventanas; con instrucciones claras y precisas sobre el uso de las opciones que aparecen en pantalla.

El componente clave de este proyecto es la emulación de ROMS de NES, SNES, y GameBoy Advance; por lo que sin éste componente el proyecto no será aceptado. El video-evidencia debe mostrar claramente que la Raspberry Pi es capaz de emular los juegos precargados y ser operada únicamente con un *gamepad*.

Importante: El uso de distribuciones de escritorio o manejadores de ventanas superpuestas en este proyecto será penalizado severamente.

Importante: La implementación del proyecto debe ser realizada desde un sistema operativo base. El uso de proyectos existentes como *RetroPie*, *Recallbox*, *Batocera*, *Lakka*, *Pi Entertainment System*, *Skraper*, etcétera significan reprobación automática.

5.5. Distro de propósito específico

Tomando como base uno de los proyectos descritos en las secciones anteriores, i) control de invernadero ii) concentrador básico *Smart Home* iii) consola de videojuegos retro, o iv) centro multimedia, el alumno generará una distro personalizada con kernel de Linux que integre únicamente los componentes mínimos necesarios para la operación del sistema elegido, y las configuraciones requeridas para que el sistema opere con la mayor eficiencia y el menor consumo de energía posible.

El sistema operativo embebido deberá cumplir con los siguientes requisitos:

- Versión de kernel
- Módulos de kernel y controladores
- Configuraciones de scheduler GP y/o RT

- Configuraciones de energía
- Imagen de kernel (sistema de archivos raíz y tipo de imagen)
- Sistema de archivos y puntos de montaje
- Configuración de tareas y programas al arranque
- Configuración de tareas y programas de mantenimiento
- Reproducción de animación o imagen estática (ej. logotipo) personalizada

Los entregables incluyen:

- Archivo IMG compatible con [balenaEtcher](#)
- Archivos de proyecto (ej. [Yocto](#) o [Buildroot](#)).
- Archivos de configuración adicionales.
- **No incluya el código fuente base de Linux**

Nota: Todas las decisiones respecto a paquetes y configuraciones tendrán que justificarse en el trabajo escrito.

Para esta opción, el uso de máquinas virtuales no es admisible.

El componente clave de este proyecto es la la imagen (archivo `img`) generado por lo que sin éste componente el proyecto no será aceptado. El video-evidencia debe mostrar claramente que la Raspberry Pi carga la distro personalizada en segundos, mostrando la imagen animación o sonido de carga durante el proceso.

Importante: Considere que la imagen generada no necesariamente será compatible con otros modelos de la tarjeta microcontroladora. Optimice para máxima velocidad y desempeño.

5.6. Consideraciones

En caso de que no se cuente con el hardware necesario, se aconseja encarecidamente el uso de una máquina virtual con Raspbian y módulos de software que simulen los componentes de hardware. Si se utiliza una máquina virtual (arquitectura i386/ia64) tendrá que verificarse que todos los componentes instalados y desarrollados sean compatibles con la arquitectura de la Raspberry Pi (ARM) y no con Intel.

El trabajo escrito (tutorial) deberá describir el funcionamiento de los componentes críticos de software y hardware, proceso de ensamblado, instalación y configuración de los mismos, así como de su uso. La instalación del sistema operativo Raspbian, ya sea en la arquitectura física o en máquina virtual, se considera trivial y no deberá incluirse en el trabajo escrito. Trabajos escritos que detallen la instalación de un sistema Windows o distro estándar de Linux en una máquina virtual serán anulados.

El código desarrollado deberá funcionar de forma óptima en la arquitectura seleccionada con los componentes elegidos para el sistema de producción. Si no se contare con los recursos para hacer una implementación física, el sistema desarrollado bajo la arquitectura de desarrollo debería ser transferible al sistema de producción con cambios mínimos. En otras palabras, los proyectos desarrollados en máquinas virtuales deberán simular también los componentes de hardware a usar, por lo menos, a nivel de comunicaciones.

6. Entregables

Los entregables deberán presentarse empaquetados en un archivo comprimido en formato `7z`, `rar` o `zip` con la siguiente estructura:

- `doc/` Tutorial y otros documentos técnicos requeridos
- `src/` Código fuente del proyecto
- `vid/` Video o archivo de texto (`txt` o `MD`) con URL del video en línea (youtube)
- `readme.MD` Archivo readme con información del proyecto y datos de los autores
- `LICENSE` Archivo de licencia (se aconseja MIT)

El archivo comprimido deberá cumplir con las siguientes especificaciones:

- El nombre del archivo comprimido deberá ser, por ejemplo, `nn-XXXX-final.7z`, donde:
 - `nn` es el número de lista del alumno a dos dígitos forzosos (ej. 01, 02, etc.).
 - `XXXX` corresponden a los cuatro primeros caracteres de su RFC o CURP.
- El tamaño del archivo comprimido no deberá exceder los 15MB:

Nota: La información del documento deberá estar debidamente citada (se aconseja estilo de IEEE Transactions) y toda cita deberá estar referida en la bibliografía. La falta de citas será considerada como plagio. A este propósito, wikipedia no es una fuente de información aceptable.

Nota: Los archivos dañados serán considerados como proyectos no entregados.

Importante: No se aceptan proyectos que no cumplan con las especificaciones anteriores.